Who Wrote It?

Identifying LLM Families from Their Responses

Nakul Bhatia | CMPSC 448 | October 11, 2026

Midterm project report | RQ1 and RQ2

Abstract

This project investigates whether a convolutional neural network (CNN) and a recurrent neural network implemented as an LSTM can identify the LLM family responsible for a response. A balanced dataset contains 1,500 examples labeled GPT, Claude, or Gemini, with 500 examples per label. Normalized prompts were grouped before an approximately 80/10/10 train/validation/test split. Six experiments compare prompt-only, response-only, and combined inputs. On 152 test examples, the response-only CNN achieved 72.37% accuracy, compared with 54.61% for the LSTM. Adding the prompt increased CNN accuracy to 75.00% but reduced LSTM accuracy to 50.00%. Prompt-only accuracy was 34.87% for CNN and 38.16% for LSTM. These results suggest that responses provide most of the useful classification signal in this dataset. They do not establish universal model fingerprints or a reliable benefit from adding prompts.

1. Problem definition

The task is supervised, three-class classification: given text associated with an LLM interaction, predict its recorded family label. RQ1 asks whether generated responses alone identify the family. RQ2 asks whether the original user prompt provides useful information, either alone or alongside the response. Both CNN and LSTM models are evaluated under all three input conditions.

The target is a family label rather than an individual model version. Multiple versions may share one label, so the results describe identification within this collected dataset, not identification of every model in a family. Domain transfer and causal feature attribution were not tested; this report does not claim completion of optional RQ3 or RQ4.

2. Dataset construction

The source is lmarena-ai/arena-human-preference-55k on Hugging Face [1], loaded with the train split using the datasets library. The preparation log records 57,477 rows and nine columns: an interaction ID, two model names, a prompt, two responses, and three preference indicators. The classifier uses text and family labels, not preference outcomes. The prepared file retains conversation_id, LLM_name, model_version, LLM_Input, and LLM_output. All five columns have zero missing values in the final 1,500-row dataset.

The log reports 43,833 extracted candidate examples and 36,341 examples after filtering: GPT 22,727, Claude 11,727, and the label Gemini 1,887. Sampling produced 500 examples per class. Only single-turn conversations with nonempty string prompts and responses are retained. Duplicate (family, prompt, response) triples are removed; prompts must contain at least 3 whitespace-delimited words and responses at least 30. Sampling and final shuffling use seed 42. GPT and Claude are selected by their name prefixes; only gemini-prefixed models receive the Gemini label. PaLM is excluded. No new responses were generated for this project.

Data preparation and experimental design

2.1 Split and leakage control

Prompts were stripped of leading/trailing whitespace, lowercased, and normalized by collapsing repeated whitespace. GroupShuffleSplit first held out approximately 20% of prompt groups, then divided that held-out portion into validation and test groups. Both calls used random_state=42. This keeps exact normalized prompt matches in the same partition. Grouping explains why the row counts are not exactly 1,200/150/150. An audit of the uploaded CSVs found zero normalized prompt overlap and zero normalized response overlap between each pair of partitions. Paraphrase overlap was not tested.

Partition	GPT	Claude	Gemini	Total
Training	389	400	407	1196
Validation	55	53	44	152
Test	56	47	49	152
Total	500	500	500	1500

2.2 Text representation

The shared preprocessing code tokenizes lowercase text into words and punctuation. A vocabulary is built only from training prompts and training responses, with a maximum of 20,000 entries including reserved PAD, UNK, and SEP tokens. Text becomes token IDs, with unknown tokens mapped to UNK and shorter sequences padded. Response-only inputs allow up to 256 tokens. Family IDs are GPT=0, Claude=1, and Gemini=2.

In run_rq2.py, combined input contains the first min(prompt length, 64) prompt tokens, the reserved SEP ID, and up to 255 minus that prompt length response tokens. Short prompts therefore leave more response space; a 64-token prompt leaves 191 response tokens. It changes the available response context relative to output-only classification, so its effect cannot be attributed solely to adding the prompt.

3. Models and training

CNN: A randomly initialized, trainable 20,000 by 128 embedding layer feeds three Conv1d branches with kernel widths 3, 4, and 5. Each branch has 64 filters, stride 1, and no convolution padding. ReLU and global max pooling produce 64 features per branch. Concatenation yields 192 features, followed by dropout 0.5 and a 192-to-3 linear layer. PAD embeddings are zero, but convolution/pooling windows are not masked; sequence-length artifacts remain a possible signal.

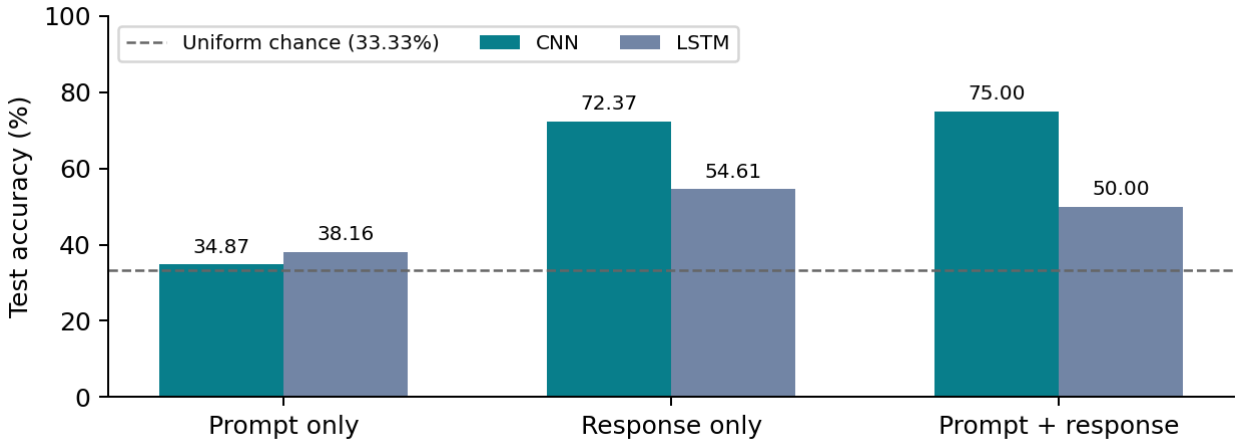
LSTM: A separate trainable 128-dimensional embedding layer feeds a one-layer, unidirectional LSTM with hidden size 64. pack_padded_sequence excludes trailing padding. The final hidden state is passed through dropout 0.5 and a 64-to-3 linear layer. LSTM gradient norms are clipped to 1.0. Neither model uses pretrained embeddings.

Both models use cross-entropy loss and Adam with learning rate 0.001, default Adam settings, batch size 32, shuffled training batches, and 10 epochs. Python, NumPy, and PyTorch seeds are set to 42, reset for each RQ2 experiment. Training used Apple MPS. The checkpoint is saved only when validation accuracy strictly improves and reloaded before testing; ties retain the earlier checkpoint. Evaluation disables dropout. No early stopping, learning-rate schedule, or hyperparameter sweep is implemented. Seeds do not guarantee identical results across hardware or library versions.

Accuracy and macro F1 are evaluated on the same 152 test examples. Macro F1 averages class F1 scores equally. Report values are calculated from the supplied confusion matrices, correcting rounded output-only entries in all_results.csv.

Results: identifying the model family

Model	Text supplied	Correct / 152	Accuracy	Macro F1
CNN	Prompt only	53	34.87%	0.3458
LSTM	Prompt only	58	38.16%	0.3817
CNN	Response only	110	72.37%	0.7260
LSTM	Response only	83	54.61%	0.5475
CNN	Prompt + response	114	75.00%	0.7523
LSTM	Prompt + response	76	50.00%	0.5010



4. RQ1: Can responses identify their generating LLM?

Within this held-out dataset, both response-only models exceed the expected 33.33% accuracy of uniform random guessing. CNN correctly classifies 110 examples and LSTM correctly classifies 83. The difference is 27 examples, or 17.76 percentage points. This supports the usefulness of response text under the tested conditions, with CNN performing better in this run.

A class-frequency baseline is also useful. Always predicting Gemini, the most frequent training label, yields 49/152 = 32.24% test accuracy. Always predicting GPT would yield 36.84% on this test set, but choosing that label from test frequencies is a descriptive reference rather than a training-selected baseline.

5. RQ2: Does the prompt help?

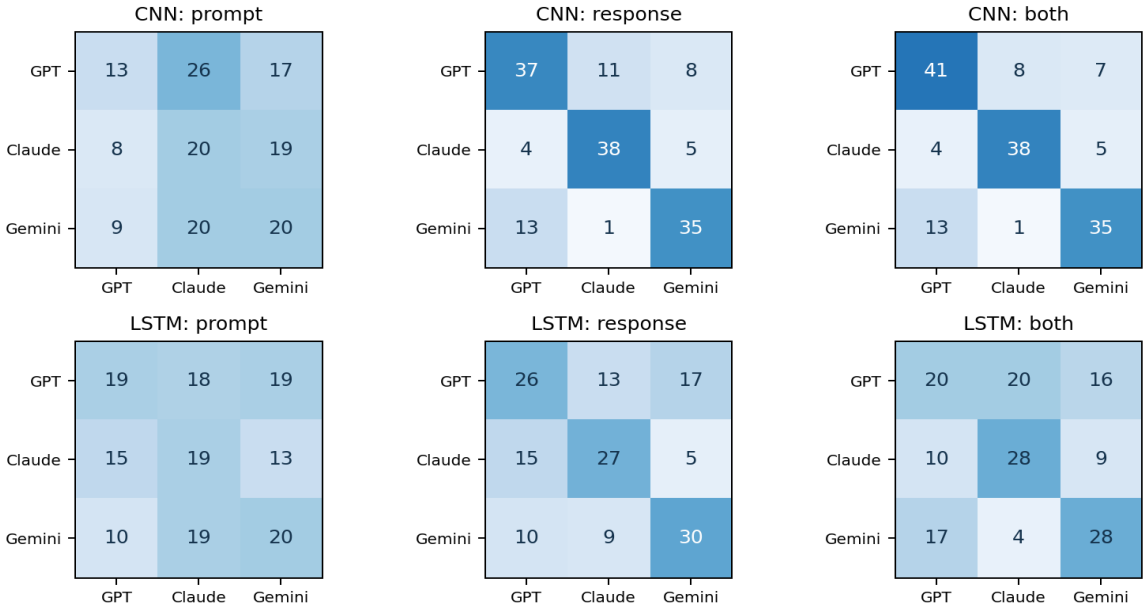
Prompt-only models correctly classify 53 examples (CNN) and 58 examples (LSTM), close to the simple baseline range. The evidence does not show strong predictive value from prompts alone. If prompts were highly predictive, possible explanations would include model-specific task selection or collection biases rather than response style.

Combining prompt and response produces 114 correct CNN predictions, a net improvement of four examples (+2.63 points). LSTM falls from 83 to 76 correct (-4.61 points). These single-run differences do not establish a consistent prompt benefit. Different response truncation budgets and random training variation are competing explanations. On the uploaded test data, output-only truncates 82 responses; combined input truncates 91 and retains fewer response tokens for those 91 examples. No repeated-seed analysis or paired significance test was performed.

Error analysis and learning behavior

6. Confusion matrices

In every matrix, rows are actual labels and columns are predictions, ordered GPT, Claude, Gemini. Each test set contains 56 GPT, 47 Claude, and 49 Gemini examples.



In the response-only CNN, Claude has the highest recall (38/47 = 80.85%), followed by Gemini (35/49 = 71.43%) and GPT (37/56 = 66.07%). Its most frequent individual confusion is Gemini predicted as GPT (13 cases). The LSTM also has its lowest response-only recall for GPT (26/56 = 46.43%), and its largest off-diagonal count is GPT predicted as Gemini (17 cases).

Adding prompts to CNN increases the GPT diagonal from 37 to 41, while Claude and Gemini diagonal counts remain 38 and 35. Aggregate matrices show where the net gain occurs, but cannot determine which individual examples switched predictions. That requires saved per-example predictions.

7. Training behavior

Run	Loss: epoch 1 to 10	Best validation accuracy
CNN, response only	1.2520 to 0.3912	78.29% (epoch 7)
LSTM, response only	1.0928 to 0.2326	54.61% (epoch 10)
CNN, prompt only	1.2485 to 0.4008	38.82% (epoch 1)
LSTM, prompt only	1.1068 to 0.2556	38.82% (epoch 5)
CNN, combined	1.2415 to 0.3556	78.95% (epoch 6)
LSTM, combined	1.0981 to 0.2564	52.63% (epoch 10)

Prompt-only losses fall substantially without sustained validation improvement, consistent with fitting training-specific patterns. The response-only LSTM also reaches low training loss while validation accuracy remains moderate. These observations suggest limited generalization; validation loss and training accuracy would provide a more complete overfitting diagnosis. CNN local-pattern detection is a plausible explanation for its advantage, but no feature ablation establishes the cause.

Limitations, lessons, and conclusion

8. Limitations and future work

Data provenance and labels. The dataset card lists Apache-2.0 [1]. The download script does not pin a source revision or package versions. GPT mixes eight versions, Claude four, and Gemini two; results may reflect version-specific signals. A model may directly name itself in a response, allowing easy identification without subtle stylistic inference. The preparation script does not remove model names or perform a comprehensive sensitive-content audit. Public availability alone does not establish redistribution rights.

Scope and uncertainty. The dataset has only 1,196 training examples and 152 test examples. One test prediction changes accuracy by about 0.66 percentage points. Only one run per condition is reported, and no domain-held-out evaluation was performed. Family versions and topic distributions may affect results, so performance should not be generalized to newer models or unseen domains.

Controlled comparisons. A stronger RQ2 experiment would retain exactly the same response tokens in output-only and combined conditions, then add a separately budgeted prompt. Repeated seeds, validation-based selection, and paired test predictions would make model comparisons more informative. Removing explicit model names or formatting would help determine which signals drive classification; these are proposed follow-ups, not completed experiments.

9. Lessons and experience

This project illustrates that preprocessing and evaluation design are as important as the network architecture. Building the vocabulary on training data avoids vocabulary leakage, while grouping prompts reduces overlap across evaluation partitions. Keeping validation and test roles separate is necessary for credible performance estimates.

The workflow also exposed practical implementation issues. A training-loss calculation incorrectly accessed .dataset on a pandas DataFrame; using train_loader.dataset fixed that error. Separate checkpoint names are needed for each model and input mode. Reserved separator tokens must survive tokenization, and LSTM sequence lengths must be handled correctly. Finally, lower training loss does not by itself establish better generalization: the LSTM reached lower response-only training loss than the CNN but lower test accuracy.

10. Conclusion

The response-only CNN achieved 72.37% accuracy and outperformed the response-only LSTM at 54.61% in this experiment. Prompt-only classification provided limited signal, and adding prompts had mixed effects: CNN rose to 75.00%, while LSTM fell to 50.00%. The strongest supported conclusion is that response text is useful for predicting the recorded family labels in this dataset. The findings do not yet isolate a universal stylistic fingerprint or a reproducible advantage from prompt inclusion.

Evidence and assistance

The report uses the course handout [2], uploaded preparation/training scripts and CSV files, and the six supplied training logs and confusion matrices. Dataset counts and cross-partition overlaps were independently checked; neural-network training and checkpoint inference were not rerun during report preparation. ChatGPT assisted with code/debugging, interpretation, and report drafting. The author is responsible for understanding and verifying the submission.

Dataset composition and reproducibility

Appendix A. Model versions in the balanced sample

Family	Model versions and example counts
GPT	gpt-3.5-turbo-0125: 25; gpt-3.5-turbo-0314: 24; gpt-3.5-turbo-0613: 95; gpt-3.5-turbo-1106: 44; gpt-4-0125-preview: 12; gpt-4-0314: 71; gpt-4-0613: 97; gpt-4-1106-preview: 132
Claude	claude-1: 145; claude-2.0: 73; claude-2.1: 165; claude-instant-1: 117
Gemini	gemini-pro: 230; gemini-pro-dev-api: 270

These counts come directly from llm_dataset.csv. There are 500 examples per family, zero missing values in the five retained columns, and no duplicate (family, prompt, response) triples. The broad family labels conceal unequal representation of individual versions; generalization to an entirely new version is not established.

Appendix B. Reproduction sequence

Run the following scripts from the project directory in order, using an environment with datasets, pandas, NumPy, scikit-learn, and PyTorch installed. The supplied CSVs and vocabulary preserve the data used for the recorded experiments; a fresh unpinned upstream download may change future samples.

Order	Script	Purpose
1	prepare_data.py	Download, filter, balance, and save llm_dataset.csv.
2	split_data.py	Create grouped train.csv, val.csv, and test.csv.
3	preprocess.py	Build the training-only vocab.json.
4	train_cnn.py	Train and evaluate the response-only CNN.
5	train_lstm.py	Train and evaluate the response-only LSTM.
6	run_rq2.py	Run prompt-only and combined conditions for both models.

run_rq2.py writes rq2_results.csv and all_results.csv. It appends fixed historical RQ1 values rather than reading newly produced output-only results, so those appended values must be refreshed after a rerun. The report corrects their precision from the original confusion matrices. Checkpoints are separate for each experiment. No new model results were introduced while preparing this report.

References

[1] LMArena. arena-human-preference-55k. Hugging Face dataset repository. Dataset card lists Apache-2.0. Accessed October 11, 2026. https://huggingface.co/datasets/lmarena-ai/arena-human-preference-55k

[2] CMPSC 448. Who Wrote It? Identifying LLMs from Their Responses. Midterm project handout, modified September 17, 2026.